



Security Audit

Aqualis (ERC20)

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Intended Smart Contract Behaviours	9
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Audit Findings	13
Centralisation	39
Conclusion	40
Our Methodology	41
Disclaimers	43
About Hashlock	44

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The Aqualis Protocol team partnered with Hashlock to conduct a security audit of their AqualisSP.sol, AqualisSPProtocolRevenue.sol, AqualisSPToken.sol, and AqualisStablecoinProcessor.sol smart contracts. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed contracts were secure. Aqualis Protocol also employs the use of a fork of Aave v3. Since no changes were made on the Aave v3 fork, only the integrations with Aave v3 from the Aqualis Protocol side have been thoroughly audited.

Project Context

Aqualis Protocol is a multi-functional DeFi protocol with both trading and lending capabilities. What sets Aqualis Protocol apart from existing DEXs is the ability to unlock liquidity locked in a DEX liquidity pool for use in lending, without impacting the trading efficiency. This process is powered by the Aqualis Protocol asset multi-utilization (AMU) algorithm, which allocates under-utilized stablecoin DEX liquidity in the Aqualis Protocol SP as capital for over-collateralized lending liquidity. Effectively, this means users have access to more capital for loans at lower interest rates, while traders can enjoy lower fees while depositors can earn higher yield farming rewards.

With the integration of AI, the AMU will also be able to support any crypto and token in addition to just stablecoins, allowing Aqualis to build the next generation of DEXs, where all liquidity pool positions are earning even if no trades are happening!"

Project Name: Aqualis Protocol

Compiler Version: ^0.8.20

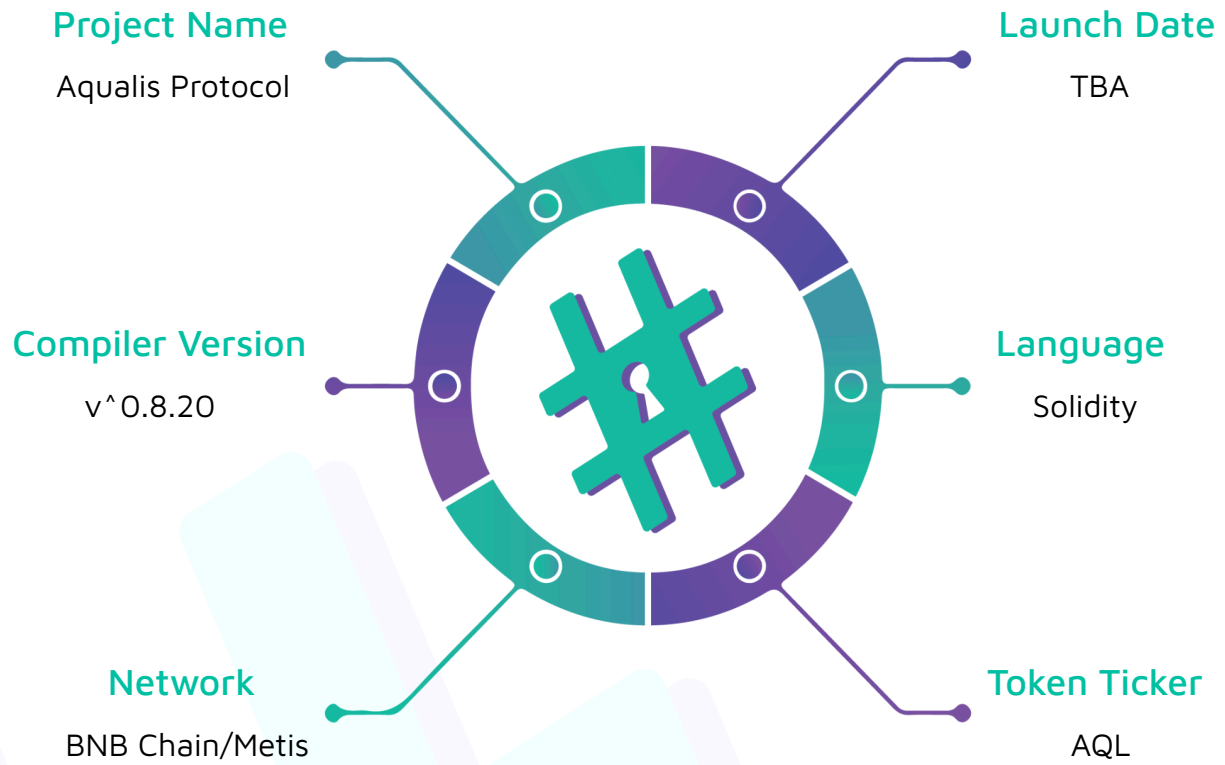
Website: www.aqualis.ai

Logo:



#Hashlock.

Hashlock Pty Ltd

Visualised Context:

Project Visuals:

Unlock DEX Liquidity with the Novel AMU AI Algorithm

The Aqualis asset multi-utilization AI (AMU AI) algorithm enables up to 80% of DEX liquidity to be simultaneously used as capital for crypto backed loans, drastically increasing lending availability in DeFi without sacrificing trading liquidity.

Aqualis is a multi-functional DeFi Protocol

Keep Assets in Crypto

Gain access to immediate liquidity from crypto holdings without selling, providing users with the funds they need without compromising their market exposure.



Optimized Asset Utilization

Increased efficiency delivers low fees and high deposit rates by maximizing asset usage through the Aqualis AMU AI algorithm, the backbone of the entire Aqualis ecosystem.



AQL Staking

Be part of a unique time-weighted staking protocol and be exponentially rewarded for commitment to the long-term vision of Aqualis.



Audit scope

We at Hashlock audited the solidity code within the Aqualis Protocol project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Aqualis Protocol Smart Contracts
Platform	Polygon / Solidity
Audit Date	August, 2024
Contract 1	AqualisSP.sol
Contract 1 MD5 Hash	16e57d6bcc427cad5c8b31d215d13454
Contract 2	AqualisSPProtocolRevenue.sol
Contract 2 MD5 Hash	30310bfc871a7ca83daaa95e0bcf2bcc
Contract 3	AqualisSPToken.sol
Contract 3 MD5 Hash	dd97091804841898d8475c0c84123c78
Contract 4	AqualisStablecoinProcessor.sol
Contract 4 MD5 Hash	6c7a7646d25103be3b36db3d5a0fe83b
GitHub Commit Hash	ec6d42272b5ed0eb4f18f70edc7f438a2cca64bd

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Hashlocked"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section.

We initially identified some significant vulnerabilities that have **since been resolved**.

Hashlock found:

3 High-severity vulnerabilities

3 Medium-severity vulnerabilities

3 Low-severity vulnerabilities

6 Gas Optimisations

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Participate in Aqualis – Bug Bounty Program: <https://hashlock.com/bug-bounty/aqualis>

Intended Smart Contract Behaviours

Claimed Behaviour	Actual Behaviour
AqualisSP.sol - Allows users to: - Swap stablecoins - Deposit stablecoins and mint SP tokens - Withdraw stablecoins by burning SP tokens - Allows admin to: - Set lending contract address - Set AMU thresholds - Set AMU rates - Allows DAOSigners to: - Set swap fee components - Set liquidity fee components	Contract achieves this functionality.
AqualisSPProtocolRevenue.sol - Protocol revenue handler abstract contract - Allows admin to: - Distribute protocol revenue - Allows DAOSigners to: - Set protocol revenue percentage - Set protocol revenue distribution percentages - Set protocol revenue distribution addresses	Contract achieves this functionality.
AqualisSPToken.sol - ERC20 token - Ownable	Contract achieves this functionality.
AqualisStablecoinProcessor.sol	Contract achieves this

- | | |
|--|--|
| <ul style="list-style-type: none">- Stablecoin addresses processor abstract contract.- Keeps an enumerable set of designated Stablecoin addresses as well as a function to get total balance. | |
|--|--|

functionality.

Code Quality

This Audit scope involves the smart contracts of the Aqualis Protocol project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Aqualis Protocol projects smart contract code in the form of GitHub access.

As mentioned above, code parts are well-commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments help understand the overall architecture of the protocol.

Dependencies

Per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry-standard open-source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies

Audit Findings

High

[H-01] AqualisSP::redeemSP - Calculation breaks when deposited stablecoin pools are unbalanced

Description

The `createSP` function allows users to deposit stablecoins and mint SP tokens depending on the amount deposited minus the fees. Users can then use the `redeemSP` function to burn their SP tokens, receiving any valid stablecoin. However, when the stablecoin pools are unbalanced the calculation breaks, leading to users losing funds.

Vulnerability Details

The `redeemSP` function shown below allows users to burn their SP tokens and receive any valid stablecoin in return.

```
function redeemSP(address _output, uint256 _amount) external watchTVL {
    if (!_stablecoins.contains(_output)) revert Errors.InvalidToken();
    if (_amount < 20e18) revert Errors.InsufficientAmount();
    (int128 fee, int128 newLiquidity) = getLiquidityFee(_output, _amount, true);

    int128 outputAmount = _amount.divu(1e18).sub(fee).mul(getSharePrice());
    uint256 transferAmount = outputAmount.mulu(10 ** _decimals[_output]);
    uint256 availableLiquidity = IERC20(_output).balanceOf(address(this));
    if (transferAmount > availableLiquidity) revert Errors.InsufficientLiquidity(_output);

    SPtoken.burn(msg.sender, _amount);
    IERC20(_output).safeTransfer(msg.sender, transferAmount);

    _accumulateFees(fee);
}
```

```

    /// @dev sync stablecoin balances and run depeg check
    _depegCheck(_output, newLiquidity.div(_syncStablecoinBalances()));

    emit SPRedeemed(msg.sender, _output, transferAmount, _amount);
}

```

This function will calculate the `fee` and `newLiquidity` in the `getLiquidityFee` function shown below:

```

function getLiquidityFee(
    address _stablecoin,
    uint256 _amount,
    bool _isExit
)
    public
    view
    returns (int128 fee, int128 newLiquidityFixed)
{
    uint256 decimals = _decimals[_stablecoin];
    uint256 amountDecimals = _isExit ? 18 : decimals;
    int128 amountFixed = _amount.divu(10 ** amountDecimals);
    uint256 initialLiquidity = IERC20(_stablecoin).balanceOf(address(this));
    if (initialLiquidity == 0) return (amountFixed.mul(liquidityBaseFee),
    amountFixed);

    int128 initialLiquidityFixed = initialLiquidity.divu(10 ** decimals);

    int128 totalStablecoinBalance = totalStablecoinBalance.add(totalSuppliedToLending);
    int128 totalLiquidityFixed = totalLiquidityFixed;

    int128 newTotalLiquidityFixed;
    if (!_isExit) {
        newLiquidityFixed = initialLiquidityFixed.add(amountFixed);
        newTotalLiquidityFixed = totalLiquidityFixed.add(amountFixed);
    } else {
        newLiquidityFixed = initialLiquidityFixed.sub(amountFixed);
    }
}

```

```

        newTotalLiquidityFixed = totalLiquidityFixed.sub(amountFixed);
    }

    int128 initialRatioFixed = initialLiquidityFixed.div(totalLiquidityFixed);
    int128 newRatioFixed = newLiquidityFixed.div(newTotalLiquidityFixed);

    int128 initialDiff =
ABDKMath64x64.abs(liquidityTargetRatio.sub(initialRatioFixed));
    int128 newDiff = ABDKMath64x64.abs(newRatioFixed.sub(liquidityTargetRatio));
    int128 avgDiff = ABDKMath64x64.avg(initialDiff, newDiff);

    int128 taxOrRebate = liquidityBaseTax.mul(avgDiff).div(liquidityTargetRatio);

    if (newDiff < initialDiff) {
        // This action benefits the Liquidity ratio, thus getting rebate instead
of tax
        fee = amountFixed.mul(liquidityBaseFee.sub(taxOrRebate));
    } else {
        // This action reduce the Liquidity ratio, thus getting taxed
        fee = amountFixed.mul(liquidityBaseFee.add(taxOrRebate));
    }
}

```

When the pools are nearly drained, this function will return a really high **fee** and a negative **newLiquidity** amount. This will result in users getting less stablecoins than they should.

Proof of Concept

Implement the following test in the **LiquidityActions.t.sol** file and run it to observe the token loss.

```

function test_03_RedeemSPIssue() external {
    uint256 USER_2USDTbalanceBefore = USDT.balanceOf(USER_2);

    //users deposit
    vm.startPrank(USER_1);

```

```

USDT.approve(address(SP), type(uint256).max);
SP.createSP(address(USDT), 30 * 1e6);
vm.stopPrank();

vm.startPrank(USER_3);
DAI.approve(address(SP), type(uint256).max);
SP.createSP(address(DAI), 1000 * 1e18);
vm.stopPrank();

vm.startPrank(USER_4);
USDC.approve(address(SP), type(uint256).max);
SP.createSP(address(USDC), 1000 * 1e6);
vm.stopPrank();

vm.startPrank(USER_2);
USDT.approve(address(SP), type(uint256).max);
SP.createSP(address(USDT), 1800 * 1e6);
vm.stopPrank();

uint256 USER_1SPbalance = SPToken.balanceOf(USER_1);
uint256 USER_2SPbalance = SPToken.balanceOf(USER_2);
uint256 USER_3SPbalance = SPToken.balanceOf(USER_3);
uint256 USER_4SPbalance = SPToken.balanceOf(USER_4);

//users redeem
vm.prank(USER_1);
SP.redeemSP(address(USDT), USER_1SPbalance);
vm.prank(USER_3);
SP.redeemSP(address(USDT), USER_3SPbalance);
vm.prank(USER_4);
SP.redeemSP(address(DAI), USER_4SPbalance);
vm.prank(USER_2);
SP.redeemSP(address(USDT), USER_2SPbalance);

```



```

//logs

uint256 USER_2USDTbalanceAfter = USDT.balanceOf(USER_2);

uint256 USER_2USDTLostAmount = USER_2USDTbalanceBefore -
USER_2USDTbalanceAfter;

console2.log("USER2 USDT lost:", USER_2USDTLostAmount);

console2.log("USER2 SP balance after", SPToken.balanceOf(USER_2));

}

```

This test will print the following logs:

```

USER2 USDT lost: 1554151524
USER2 SP balance after 0

```

Showing that USER_2 lost most of their USDT while burning all of their SP.
1800e18 SP burned and received 245.8e6 USDT in return.

Impact

Users will receive less than the intended amount when they call the `redeemSP` function.

Recommendation

Implement a new parameter for users to enter their minimum desired stable coin amount and revert if this condition is not met. An example shown below:

```

function redeemSP(address _output, uint256 _amount, uint256 minAmountOut) external
watchTVL {
    //rest of the function

    uint256 transferAmount = outputAmount.mul(10 ** _decimals[_output]);
    if (transferAmount < minAmountOut) revert Errors.InsufficientAmount();

    //rest of the function
}

```

Status

Resolved

[H-02] AqualisSP - No way to withdraw removed stablecoins resulting in them being stuck in the contract

Description

The `removeStablecoinAddress` function allows the admin to remove an already added stablecoin from the set of stablecoins that are allowed to be used in the contract. When a stablecoin is removed it can no longer be used in the contract's functionality. After removal, there are no ways to interact with these tokens and they will be stuck in the contract.

Vulnerability Details

The `removeStablecoinAddress` function shown below allows the admin to remove a stablecoin:

```
function removeStablecoinAddress(address _stablecoinAddress) external onlyAdmin {
    if (_stablecoinAddress == address(0)) revert Errors.ZeroAddress();
    _removeStablecoin(_stablecoinAddress);
}

function _removeStablecoin(address _stablecoin) internal {
    if (_stablecoins.remove(_stablecoin)) {
        _decimals[_stablecoin] = 0x0;
        _recalculateTargetRatio();
        emit StablecoinRemoved(_stablecoin);
    }
}
```

When a stablecoin is removed, it will not be able to be used in deposit, withdraw or swap functions.

Impact

Stablecoin that is removed will not be intractable with or withdrawable from the contract until it is added again resulting in these coins being stuck in the contract.

Recommendation

Implement a new withdraw function that is only able to withdraw removed stablecoins to a DAO or Admin address. However, be aware of the centralization risks that come with existence of this function and take proper measures to make sure it can not be used maliciously. The DAO or the Admin should supply the withdrawn stablecoin amount in a valid stable coin with this function.

Status

Resolved

[H-03] AqualisSP::removeStablecoinAddress - Removing a stablecoin breaks calculations

Description

The `removeStablecoinAddress` function allows the admin to remove an already added stablecoin from the set of stablecoins that are allowed to be used in the contract. When a stablecoin is removed it can no longer be used in the contracts functionality.

The `redeemSP` function allows users to withdraw their choice of stablecoins while burning their SP tokens. However, when a stablecoin is removed, the user that redeems first will get the stablecoin amount that they should. This first redeem will sync stablecoin balances and subsequent users will receive less stablecoins when they redeem.

Vulnerability Details

The `removeStablecoinAddress` function shown below allows the admin to remove a stablecoin:

```
function removeStablecoinAddress(address _stablecoinAddress) external onlyAdmin {
    if (_stablecoinAddress == address(0)) revert Errors.ZeroAddress();
    _removeStablecoin(_stablecoinAddress);
}

function _removeStablecoin(address _stablecoin) internal {
    if (_stablecoins.remove(_stablecoin)) {
```

```

        _decimals[_stablecoin] = 0x0;
        _recalculateTargetRatio();
        emit StablecoinRemoved(_stablecoin);
    }
}

```

When a stablecoin is removed, it will not be able to be used in deposit, withdraw or swap functions.

The `redeemSP` function shown below allows users to withdraw stablecoins:

```

function redeemSP(address _output, uint256 _amount) external watchTVL {
    if (!_stablecoins.contains(_output)) revert Errors.InvalidToken();
    if (_amount < 20e18) revert Errors.InsufficientAmount();
    (int128 fee, int128 newLiquidity) = getLiquidityFee(_output, _amount, true);

    int128 outputAmount = _amount.divu(1e18).sub(fee).mul(getSharePrice());
    uint256 transferAmount = outputAmount.mulu(10 ** _decimals[_output]);
    uint256 availableLiquidity = IERC20(_output).balanceOf(address(this));
    if (transferAmount > availableLiquidity) revert
    Errors.InsufficientLiquidity(_output);

    SPtoken.burn(msg.sender, _amount);
    IERC20(_output).safeTransfer(msg.sender, transferAmount);

    _accumulateFees(fee);

    /// @dev sync stablecoin balances and run depeg check
    _depegCheck(_output, newLiquidity.div(_syncStablecoinBalances()));

    emit SPRedeemed(msg.sender, _output, transferAmount, _amount);
}

```

When a stablecoin is removed and the stablecoin balances in the contract are synced due to the current imbalance in minted SP and stablecoin balances, the worth of an SP token will be lowered and users will be receiving less stablecoins than they should when

they redeem.

If a user redeems before the sync of balances, when they redeem they will get the stablecoin amount they should and this will sync the stablecoin balances in the contract. Subsequent redeems will cause users to receive less stablecoins than they should.

Proof of Concept

Implement the following test in [LiquidityActions.t.sol](#) file, run it and observe the logs to notice the loss of tokens.

```
function test_04_RedeemSPAAfterRemoval() external {
    uint256 USER_1USDTBefore = USDT.balanceOf(USER_1);
    uint256 USER_3USDCBefore = USDC.balanceOf(USER_3);

    //users deposit
    vm.startPrank(USER_1);
    USDT.approve(address(SP), type(uint256).max);
    SP.createSP(address(USDT), 1000 * 1e6);
    vm.stopPrank();

    vm.startPrank(USER_3);
    USDC.approve(address(SP), type(uint256).max);
    SP.createSP(address(USDC), 1000 * 1e6);
    vm.stopPrank();

    vm.startPrank(USER_4);
    DAI.approve(address(SP), type(uint256).max);
    SP.createSP(address(DAI), 1000 * 1e18);
    vm.stopPrank();

    //admin removes DAI
    vm.startPrank(ADMIN);
    SP.removeStablecoinAddress(address(DAI));
    vm.stopPrank();

    uint256 USER_1SPBalance = SPToken.balanceOf(USER_1);
    uint256 USER_3SPBalance = SPToken.balanceOf(USER_3);
```

```
//users redeem  
  
vm.startPrank(USER_1);  
SP.redeemSP(address(USDT), USER_1SPBalance);  
vm.stopPrank();  
  
vm.startPrank(USER_3);  
SP.redeemSP(address(USDC), USER_3SPBalance);  
vm.stopPrank();  
  
uint256 USER_1USDTAfter = USDT.balanceOf(USER_1);  
uint256 USER_3USDCAfter = USDC.balanceOf(USER_3);  
console2.log("User1 USDT lost:", USER_1USDTBefore - USER_1USDTAfter);  
console2.log("User3 USDC lost:", USER_3USDCBefore - USER_3USDCAfter);  
}
```

The test will print out the following logs:

```
User1 USDT lost: 458273  
User3 USDC lost: 499886974
```

As seen from the test logs, USER_1 received the stablecoin amount that they should since the balances are not synced. USER_3 receives less stablecoin since the balances are synced at this point.

Impact

When a stablecoin is removed and this is not compensated in any way by adding a new stablecoin with the same amount, users will end up receiving less stablecoins than they should. The first user to redeem will receive the amount that they should and subsequent users will receive less tokens due to the stablecoin balances in the contract being synced after the first redeem.

Recommendation

When a stablecoin is removed the DAO or the Admin should supply the removed stablecoin's amount in a valid stable coin to keep contract functionality working as intended. Removal and addition should be done in one transaction and ideally in a single function.

Status

Resolved

Medium

[M-01] AqualisSP::createSP, _stableSwap - Functions are vulnerable to fee-on-transfer accounting-related issues

Description

The functions below transfer funds from the caller to the receiver via `transferFrom()`, but do not ensure that the actual number of tokens received is the same as the input amount to the transfer. If the token is a fee-on-transfer token, the balance after the transfer will be smaller than expected, leading to accounting issues.

Vulnerability Details

The `createSP` function shown below allows users to deposit an allowed stablecoin and mint SP tokens.

```
function createSP(address _input, uint256 _amount) external watchTVL {
    if (!_stablecoins.contains(_input)) revert Errors.InvalidToken();

    uint256 inputDecimals = _decimals[_input];
    int128 amountFixed = _amount.divu(10 ** inputDecimals);
    if (amountFixed < minimumStableInput) revert Errors.InsufficientAmount();

    (int128 fee, int128 newLiquidity) = getLiquidityFee(_input, _amount, false);
    /// @dev recursive depeg check
```

```

    if (isDepeg[_input]) {
        _depegCheck(_input, newLiquidity.div(_syncStablecoinBalances()));
        if (isDepeg[_input]) revert Errors.DepegProtection();
    }

    uint256 mintAmount = amountFixed.sub(fee).mul(getSharePrice()).mulu(1 ether);

    IERC20(_input).safeTransferFrom(msg.sender, address(this), _amount);
    SPtoken.mint(msg.sender, mintAmount);

    _accumulateFees(fee);

    /// @dev sync stablecoin balances and run depeg check
    _depegCheck(_input, newLiquidity.div(_syncStablecoinBalances()));

    emit SPCreated(msg.sender, _input, _amount, mintAmount);
}

```

The highlighted part transfers the stablecoin from the user to the contract and mints them equal amounts of SP tokens. However, since the amount that is received from the transfer isn't checked, if a fee on transfer token is used as the stablecoin of choice, the contract will end up receiving less tokens than it should while minting the full amount. The same vulnerability exists in the `_stableSwap` function that allows users to swap stablecoins.

It should be noted that only admin can add valid stablecoins that can be used, the admin must add a fee on transfer token as a stablecoin for this vulnerability to exist.

Impact

Protocol will end up receiving less than the intended amount when fee on transfer tokens are used.

Recommendation

Implement checks in your functions to calculate the received token amount and use this amount in your calculations, an example shown below:




```
function createSP(address _input, uint256 _amount) external watchTVL {
    //rest of the function
    uint256 balanceBefore = IERC20(_input).balanceOf(address(this));
    IERC20(_input).safeTransferFrom(msg.sender, address(this), _amount);
    uint256 balanceAfter = IERC20(_input).balanceOf(address(this));
    //rest of the function
}
```

Status

Resolved

[M-02] AqualisSP::redeemSP - Users deposited stablecoins will be stuck in the contract due to redeem limits

Description

The `createSP` function allows users to deposit stablecoins and mint SP tokens depending on the amount deposited minus the fees. Users can then use the `redeemSP` function to burn their SP tokens, receiving any valid stablecoin. However, due to the minimum redeem limit, user's tokens will be stuck in the contract.

Vulnerability Details

The `createSP` function shown below allows users to deposit any amount that is greater than or equal to the `minimumStableInput` variable which is set to 20 USD worth of tokens.

```
function createSP(address _input, uint256 _amount) external watchTVL {
    if (!_stablecoins.contains(_input)) revert Errors.InvalidToken();

    uint256 inputDecimals = _decimals[_input];
    int128 amountFixed = _amount.divu(10 ** inputDecimals);
    if (amountFixed < minimumStableInput) revert Errors.InsufficientAmount();
    //rest of the function
}
```

However, due to the fees, if a user deposits for example 20e6 USDC, they will receive less than 20e18 SP tokens.

When this user tries to withdraw their stablecoins with the `redeemSP` function shown below:

```
function redeemSP(address _output, uint256 _amount) external watchTVL {
    if (!_stablecoins.contains(_output)) revert Errors.InvalidToken();
    if (_amount < 20e18) revert Errors.InsufficientAmount();
    //rest of the function
}
```

It will not be possible for this user to withdraw their stablecoins until they deposit more stablecoins due to the user having less than 20e18 SP tokens.

Proof of Concept

Implement the following test in the `LiquidityActions.t.sol` file and run it.

```
function test_03_RedeemSPRevert() external {
    //users deposit
    vm.startPrank(USER_1);
    USDT.approve(address(SP), type(uint256).max);
    SP.createSP(address(USDT), 20 * 1e6);
    vm.stopPrank();

    vm.startPrank(USER_2);
    USDC.approve(address(SP), type(uint256).max);
    SP.createSP(address(USDC), 1000 * 1e6);
    vm.stopPrank();

    vm.startPrank(USER_3);
    DAI.approve(address(SP), type(uint256).max);
    SP.createSP(address(DAI), 1000 * 1e18);
    vm.stopPrank();

    uint256 USER_1SPbalance = SPToken.balanceOf(USER_1);
}
```

```

uint256 USER_3SPbalance = SPToken.balanceOf(USER_3);

//users redeem
vm.prank(USER_1);
vm.expectRevert();
SP.redeemSP(address(USDT), USER_1SPbalance);
vm.prank(USER_3);
SP.redeemSP(address(DAI), USER_3SPbalance);
}

```

The test will revert as expected when USER_1 tries to redeem with all their SP tokens.

Impact

User's stablecoins will be stuck in the contract until they deposit more stablecoins to mint more SP tokens.

Recommendation

Change the minimum amount check in the `createSP` function as shown below to make sure users can not have less than 20e18 SP tokens.

```

function createSP(address _input, uint256 _amount) external watchTVL {
    if (!_stablecoins.contains(_input)) revert Errors.InvalidToken();

    uint256 inputDecimals = _decimals[_input];
    int128 amountFixed = _amount.divu(10 ** inputDecimals);

    (int128 fee, int128 newLiquidity) = getLiquidityFee(_input, _amount, false);
    /// @dev recursive depeg check
    if (isDepeg[_input]) {
        _depegCheck(_input, newLiquidity.div(_syncStablecoinBalances()));
        if (isDepeg[_input]) revert Errors.DepegProtection();
    }

    uint256 mintAmount = amountFixed.sub(fee).mul(getSharePrice()).mulu(1 ether);
    if (mintAmount < 20e18) revert ErrorsInsufficientAmount();
}

```

```

IERC20(_input).safeTransferFrom(msg.sender, address(this), _amount);
SPtoken.mint(msg.sender, mintAmount);

_accumulateFees(fee);

/// @dev sync stablecoin balances and run depeg check
_depegCheck(_input, newLiquidity.div(_syncStablecoinBalances()));

emit SPCreated(msg.sender, _input, _amount, mintAmount);
}

```

Status

Resolved

[M-03] AqualisSP::_stableSwap - No checks done to make sure input and output tokens are not the same

Description

The `_stableSwap` function allows users to swap their valid stablecoins for any valid stablecoins. However, this function lacks checks to make sure that the input token is not the same as the output token. Accidental user inputs will lead them to swapping the same token and paying fees for no reason.

Vulnerability Details

The `_stableSwap` function shown below allows users to swap stablecoins:

```

function _stableSwap(
    address _input,
    address _output,
    uint256 _amountIn,
    uint256 _minAmountOut,
    address _to
)

```

```

internal
{
    uint256 inputDecimals = _decimals[_input];
    uint256 outputDecimals = _decimals[_output];
    int128 inputLiquidity = getLiquidity(_input, inputDecimals);
    int128 outputLiquidity = getLiquidity(_output, outputDecimals);

    int128 inputAmount = _amountIn.divu(10 ** inputDecimals);
    if (inputAmount < minimumStableInput) revert Errors.InsufficientAmount();

    int128 fee = _getStableSwapFee(inputLiquidity, outputLiquidity, inputAmount);
    uint256 amountOut = inputAmount.sub(fee).mulu(10 ** outputDecimals);

    if (_minAmountOut != 0 && amountOut < _minAmountOut) revert
    Errors.InsufficientSlippage();

    if (inputAmount.sub(fee) > outputLiquidity) revert
    Errors.InsufficientLiquidity(_output);

    IERC20(_input).safeTransferFrom(msg.sender, address(this), _amountIn);
    address to = _to == address(0) ? msg.sender : _to;
    IERC20(_output).safeTransfer(to, amountOut);

    _accumulateFees(fee);

    /// @dev Sync stablecoin balances after swap and run depeg check
    int128 totalStableBalance = _syncStablecoinBalances();
    _depegCheck(_input, inputLiquidity.div(totalStableBalance));
    _depegCheck(_output, outputLiquidity.div(totalStableBalance));

    emit Swap(_input, _output, _amountIn, amountOut, msg.sender, to);
}

```

This internal function is called by the `swapExactStableToStable` and `swapStableToExactStable` functions shown below:

```
function swapExactStableToStable(
```



```

        address _input,
        address _output,
        uint256 _amountIn,
        uint256 _minAmountOut,
        address _to
    )
    external
    {
        if (!_stablecoins.contains(_input) || !_stablecoins.contains(_output)) {
            revert Errors.InvalidSwapPair();
        }
        _stableSwap(_input, _output, _amountIn, _minAmountOut, _to);
    }

    function swapStableToExactStable(
        address _input,
        address _output,
        uint256 _maxAmountIn,
        uint256 _amountOut,
        address _to
    )
    external
    {
        uint256 amountIn = getAmountInStable(_input, _output, _amountOut);
        if (_maxAmountIn != type(uint256).max) {
            if (amountIn > _maxAmountIn) revert Errors.InsufficientSlippage();
        }
        _stableSwap(_input, _output, amountIn, 0, _to);
    }

```

As seen in all the functions above, only parameter validation done is to make sure that the inputted stablecoins are valid stablecoins accepted by the protocol. There are no protections implemented to make sure that the input and output parameters are different.

Impact

Allowing the same token to be swapped with itself will lead to users paying fees for no reason and losing out on tokens.

Recommendation

Implement checks in the function to revert in the explained scenario. An example shown below:

```
function _stableSwap(  
    address _input,  
    address _output,  
    uint256 _amountIn,  
    uint256 _minAmountOut,  
    address _to  
)  
    internal  
{  
    if (_input == _output) revert Errors.InvalidSwapPair();  
    //Rest of the function
```

Status

Resolved

Low

[L-01] AqualisSP::createSP - Incorrect de-peg protection calculation blocks transfer of tokens even when it is within the accepted threshold

Description

The `createSP` function allows users to deposit valid stablecoins and mint SP tokens. This function has implemented checks to make sure that the deposited token will not cause it to rise to over 95% liquidity in the protocol.

```

function createSP(address _input, uint256 _amount) external watchTVL {
    //rest of the function
    /// @dev recursive depeg check
    if (isDepeg[_input]) {
        newLiquidity.div(_syncStablecoinBalances().add(amountFixed));
        if (isDepeg[_input]) revert Errors.DepergProtection();
    }
    //rest of the function
}

```

However, this check will revert transactions even if the deposited tokens liquidity will stay below 95%. User's can bypass this revert by depositing the same amount in multiple transactions.

Recommendation

Change the de-peg check so that it calculates correctly and does not revert when it shouldn't.

Status

Resolved

[L-02] AqualisStablecoinProcessor::removeStablecoinAddress - Removal of stablecoin causes the AqualisSP contract to have incorrect state variable

Description

The `removeStablecoinAddress` function allows the admin to remove a stablecoin from the contract, stopping it from being used in the `AqualisSP` contracts functionality.

```

function removeStablecoinAddress(address _stablecoinAddress) external onlyAdmin {
    if (_stablecoinAddress == address(0)) revert Errors.ZeroAddress();
    _removeStablecoin(_stablecoinAddress);
}

```


However, when a stable coin address is removed, the stablecoin amount in the contract is not synced. This causes the `totalStablecoinBalance` variable in the `AqualisSP` contract to have an incorrect value.

Recommendation

Sync the stablecoin amount in the contract to update the `totalStablecoinBalance` variable.

Alternatively, make sure that when a stablecoin is removed it is instantly compensated by adding another stablecoin with the same amount.

Status

Resolved

[L-03] Contracts - Centralization risk for privileged functions

Description

Contracts with privileged functions need the owner to be trusted not to perform malicious updates or drain funds. This may also cause a single point failure. Instances of the centralization risk shown below:

Functions in the `AqualisStablecoinProcessor` contract shown below update critical state variables, malicious or accidental updates can harm users and the protocol.

```
function addStablecoinAddress()
function removeStablecoinAddress()
```

Functions in the `AqualisSP` contract shown below update critical state variables, malicious or accidental updates can harm users and the protocol.

```
function setLendingContractAddress()
function setDepegThreshold()
```

Recommendation

To mitigate centralization risks associated with privileged functions, consider implementing a multi-signature or decentralized governance mechanism. Instead of

relying solely on a single owner/administrator, distribute control and decision-making authority among multiple parties or stakeholders.

Status

Resolved

Gas

[G-01] AqualisSP::_watchTVLLogic - State variables that are used multiple times in a function should be cached

Description

When performing multiple operations on a state variable in a function, it is recommended to cache it first. Either multiple reads or multiple writes to a state variable can save gas by caching it on the stack. Instances highlighted below:

```
function _watchTVLLogic() internal {
    if (address(lendingPool) == address(0)) return;
    int128 TVL = totalStablecoinBalance.add(totalSuppliedToLending);
    if (TVL > minTVLToEnableAMU.add(AMUThresholdAmount)) {
        int128 currRatio = totalSuppliedToLending.div(totalStablecoinBalance);
        int128 diff = ABDKMath64x64.abs(lendingToTradeRatio.sub(currRatio));
        int128 diffAmount = diff.mul(totalStablecoinBalance);
        if (currRatio < lendingToTradeRatio) {
            if (diffAmount > AMUThresholdAmount) {
                _depositToLending(AMUThresholdAmount);
            }
        } else {
            if (diffAmount > AMUThresholdAmount) {
                _withdrawFromLending(AMUThresholdAmount);
            }
        }
    } else {
```

```

        if (totalSuppliedToLending > 0 && TVL.add(AMUThresholdAmount) <
minTVLToEnableAMU) {
            _withdrawAllFromLending();
        }
    }
}

```

Recommendation

Cache state variables in stack or local memory variables within functions when they are used multiple times.

Status

Resolved

[G-02] AqualisSP, AqualisStablecoinProcessor - Increments can be unchecked in for-loops

Description

Newer versions of the Solidity compiler will check for integer overflows and underflows automatically. This provides safety but increases gas costs. When an unsigned integer is guaranteed to never overflow, the unchecked feature of Solidity can be used to save gas costs. A common case for this is for-loops using a strictly-less-than comparison in their conditional statement.

Recommendation

Use unchecked math to block overflow / underflow check to save Gas.
Alternatively, use Solidity [version ^0.8.22](#).

Status

Resolved

[G-03] AqualisSP - address(this) should be cached

Description

Caching saves gas when compared to repeating the calculation at each point it is used in the contract.

Recommendation

To enhance gas efficiency, cache the contract's address by storing address(this) in a state variable at the point of contract deployment or initialization. Use this cached address throughout the contract instead of repeatedly calling address(this). This practice reduces the gas cost associated with multiple computations of the contract's address, leading to more efficient contract execution, especially in scenarios with frequent usage of the contract's address.

Status

Resolved

[G-04] AqualisSP::setDepegThreshold - Function should revert earlier on invalid inputs

Description

The `setDepegThreshold` function shown below allows the admin to set the de-peg threshold:

```
function setDepegThreshold(uint256 _numerator, uint256 _denominator) external  
onlyAdmin {  
    depegThreshold = PercentageMath.createPercentageWithMax(_numerator,  
_denominator, _denominator);  
}
```

The function will revert if the `_numerator` is greater than the `_denominator` or if the `_denominator` is 0. However, checks can be added to optimise the revert conditions so invalid inputs revert earlier.

Recommendation

Update the function as shown below:

```
function setDepegThreshold(uint256 _numerator, uint256 _denominator) external
onlyAdmin {
    if (_numerator > _denominator || _denominator == 0) revert
    Errors.InvalidInput();
    depegThreshold = PercentageMath.createPercentageWithMax(_numerator,
    _denominator, _denominator);
}
```

Status

Resolved

[G-05] AqualisStablecoinProcessor::addStablecoinAddress - Function should revert on duplicate inputs

Description

The `addStablecoinAddress` function shown below allows the admin to add a valid stablecoin to be used by the protocol

```
function addStablecoinAddress(address _stablecoinAddress) external onlyAdmin {
    if (_stablecoinAddress == address(0)) revert Errors.ZeroAddress();
    _addStablecoin(_stablecoinAddress);
}

function _addStablecoin(address _stablecoin) internal {
    uint256 decimals = uint256(IERC20Metadata(_stablecoin).decimals());
    if (_stablecoins.add(_stablecoin)) {
        _decimals[_stablecoin] = decimals;
        _recalculateTargetRatio();
        emit StablecoinAdded(_stablecoin);
    }
}
```

```
}
```

The function will not do anything if the stablecoin address has already been added. However, checking if the value is already existing and reverting in this case will save gas.

Recommendation

Check if the address already exists in the `_stablecoins` and revert if that is the case. An example shown below:

```
function addStablecoinAddress(address _stablecoinAddress) external onlyAdmin {
    if (_stablecoinAddress == address(0)) revert Errors.ZeroAddress();
    if (_stablecoins.contains(_stablecoinAddress)) revert Errors.InvalidToken();
    _addStablecoin(_stablecoinAddress);
}
```

Status

Resolved

[G-06] AqualisStablecoinProcessor::removeStablecoinAddress - Function should revert on incorrect inputs

Description

The `removeStablecoinAddress` function shown below allows the admin to remove a stablecoin from the pool of valid stablecoins, stopping it from being used in the protocol

```
function removeStablecoinAddress(address _stablecoinAddress) external onlyAdmin {
    if (_stablecoinAddress == address(0)) revert Errors.ZeroAddress();
    _removeStablecoin(_stablecoinAddress);
}

function _removeStablecoin(address _stablecoin) internal {
    if (_stablecoins.remove(_stablecoin)) {
        _decimals[_stablecoin] = 0x0;
        _recalculateTargetRatio();
    }
}
```

```
        emit StablecoinRemoved(_stablecoin);  
    }  
}
```

The function will not do anything if the stablecoin address hasn't already been added. However, checking if the value does not exist and reverting in this case will save gas.

Recommendation

Check if the address exists in the `_stablecoins`, revert if it does not. An example shown below:

```
function removeStablecoinAddress(address _stablecoinAddress) external onlyAdmin {  
    if (_stablecoinAddress == address(0)) revert Errors.ZeroAddress();  
    if (!_stablecoins.contains(_stablecoinAddress)) revert Errors.InvalidToken();  
    _removeStablecoin(_stablecoinAddress);  
}
```

Status

Resolved

Centralisation

The Aqualis Protocol project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlocks analysis, the Aqualis Protocol project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au

#Hashlock.



#Hashlock.

Hashlock Pty Ltd